

EXPERIMENT – 13

Title

To Implement Trigger on a Table

Aim / Objective

To understand and implement **Database Triggers** in Oracle PL/SQL for automatically executing actions in response to database events such as **INSERT**, **UPDATE**, and **DELETE**.

Theory

A **Trigger** is a stored PL/SQL program that is automatically executed (or "fired") when a specified event occurs on a table or view. Triggers are commonly used to enforce business rules, maintain audit logs, validate data, and automate database operations.

Unlike procedures and functions, triggers are executed automatically without being explicitly called by the user.

Types of Triggers

1. BEFORE Trigger

Executes before an INSERT, UPDATE, or DELETE operation.

2. AFTER Trigger

Executes after an INSERT, UPDATE, or DELETE operation.

3. ROW-Level Trigger

Executes once for each row affected by the SQL statement.

4. Statement-Level Trigger

Executes once for the entire SQL statement regardless of the number of rows affected.

Trigger Events

Triggers can be fired on:

- INSERT
 - UPDATE
 - DELETE
-

Learning Objectives

After completing this experiment, students will be able to:

- Understand database triggers.
 - Create BEFORE and AFTER triggers.
 - Use OLD and NEW values.
 - Automate database operations.
 - Apply triggers to maintain data integrity.
-

Software Requirements

- Oracle Database
 - Oracle SQL Developer / SQL*Plus
-

Algorithm / Procedure

1. Open Oracle SQL Developer.
 2. Connect to the Oracle database.
 3. Create Employee and Audit tables.
 4. Insert sample records.
 5. Create triggers.
 6. Perform INSERT, UPDATE, and DELETE operations.
 7. Observe the trigger execution.
 8. Stop.
-

Program

Step 1 : Create Employee Table

```
CREATE TABLE Employee
(
EmpID NUMBER PRIMARY KEY,
EmpName VARCHAR2(30),
Department VARCHAR2(30),
Salary NUMBER(10,2)
);
```

Step 2 : Insert Sample Records

```
INSERT INTO Employee VALUES
(101,'Rahul','IT',35000);
```

```
INSERT INTO Employee VALUES
(102,'Priya','HR',42000);
```

```
COMMIT;
```

Step 3 : Create Audit Table

```
CREATE TABLE Employee_Audit
(
AuditID NUMBER GENERATED BY DEFAULT AS IDENTITY,
EmpID NUMBER,
Operation VARCHAR2(20),
ActionDate DATE
);
```

Program 1 : AFTER INSERT Trigger

```
CREATE OR REPLACE TRIGGER TRG_Insert

AFTER INSERT

ON Employee

FOR EACH ROW

BEGIN

INSERT INTO Employee_Audit

(EmpID,Operation,ActionDate)

VALUES

(:NEW.EmpID,'INSERT',SYSDATE);

END;
```

/

Test Trigger

```
INSERT INTO Employee  
VALUES  
(103,'Aman','Finance',39000);  
COMMIT;
```

Verify

```
SELECT * FROM Employee_Audit;
```

Expected Output

	AuditID	EmpID	Operation	ActionDate
1	103	INSERT	Current Date	

Program 2 : AFTER UPDATE Trigger

```
CREATE OR REPLACE TRIGGER TRG_Update  
AFTER UPDATE  
ON Employee  
FOR EACH ROW  
BEGIN  
INSERT INTO Employee_Audit  
(EmpID,Operation,ActionDate)  
VALUES  
(:NEW.EmpID,'UPDATE',SYSDATE);  
END;  
/
```

Test Trigger

```
UPDATE Employee  
SET Salary=45000  
WHERE EmpID=101;  
COMMIT;
```

Verify

```
SELECT * FROM Employee_Audit;
```

Expected Output

AuditID	EmpID	Operation
---------	-------	-----------

1	103	INSERT
2	101	UPDATE

Program 3 : AFTER DELETE Trigger

```
CREATE OR REPLACE TRIGGER TRG_Delete  
AFTER DELETE  
ON Employee  
FOR EACH ROW  
BEGIN  
INSERT INTO Employee_Audit  
(EmpID,Operation,ActionDate)  
VALUES  
(:OLD.EmpID,'DELETE',SYSDATE);  
END;  
/
```

Test Trigger

```
DELETE FROM Employee
WHERE EmpID=102;
COMMIT;
```

Verify

```
SELECT * FROM Employee_Audit;
```

Expected Output

AuditID	EmpID	Operation
1	103	INSERT
2	101	UPDATE
3	102	DELETE

Program 4 : BEFORE INSERT Trigger

The following trigger prevents insertion of employees with a salary less than ₹10,000.

```
CREATE OR REPLACE TRIGGER TRG_CheckSalary
BEFORE INSERT
ON Employee
FOR EACH ROW
BEGIN
IF :NEW.Salary<10000 THEN
RAISE_APPLICATION_ERROR
(-20001,'Salary must be greater than 10000');
END IF;
END;
/
```

Test Trigger

```
INSERT INTO Employee
```

```
VALUES
```

```
(104,'Neha','HR',8000);
```

Expected Output

ORA-20001: Salary must be greater than 10000

Program 5 : Display Employee Table

```
SELECT * FROM Employee;
```

Entity Relationship Diagram

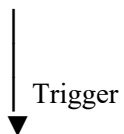
Employee

EmpID (PK)

EmpName

Department

Salary



Employee_Audit

AuditID (PK)

EmpID

Operation

ActionDate

Flowchart

Start



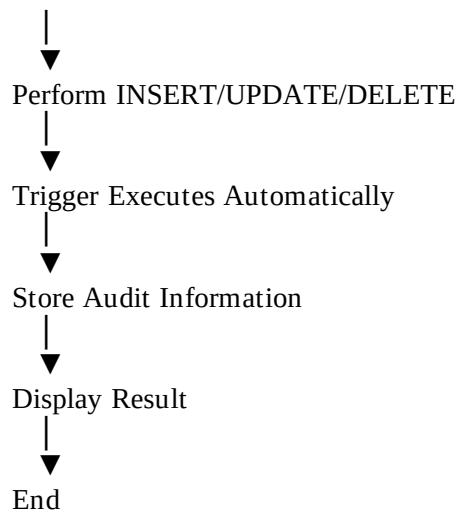
Create Employee Table



Create Audit Table



Create Trigger



Explanation

CREATE TRIGGER

Creates a database trigger.

BEFORE Trigger

Executes before the DML operation.

AFTER Trigger

Executes after the DML operation.

FOR EACH ROW

Creates a row-level trigger that executes once for every affected row.

:NEW

Contains the new values during INSERT or UPDATE operations.

:OLD

Contains the old values during UPDATE or DELETE operations.

RAISE_APPLICATION_ERROR

Generates a user-defined error message.

Observation

- The INSERT trigger automatically stored audit information.
 - The UPDATE trigger recorded modified records.
 - The DELETE trigger logged deleted records.
 - The BEFORE trigger prevented invalid salary insertion.
 - The audit table maintained a complete history of database operations.
-

Result

Database triggers were successfully created and implemented. The triggers automatically executed during INSERT, UPDATE, and DELETE operations, demonstrating how triggers can enforce business rules and maintain audit information in Oracle databases.

Applications

- Banking Systems
 - Payroll Management
 - Hospital Management Systems
 - Inventory Management Systems
 - Student Information Systems
 - Audit Logging
 - Security Monitoring
-

Advantages

- Executes automatically.
- Maintains data integrity.
- Enforces business rules.
- Records audit information.

- Reduces application coding.
 - Improves database security.
-

Limitations

- May reduce performance if too many triggers exist.
 - Difficult to debug.
 - Hidden execution can make maintenance complex.
 - Excessive trigger usage may complicate database design.
-

Viva Questions

1. What is a trigger?
 2. What is the difference between a trigger and a stored procedure?
 3. What are BEFORE and AFTER triggers?
 4. What is the purpose of FOR EACH ROW?
 5. What are :NEW and :OLD values?
 6. What is a row-level trigger?
 7. What is a statement-level trigger?
 8. What is RAISE_APPLICATION_ERROR?
 9. What are the applications of triggers?
 10. What are the advantages and disadvantages of triggers?
-

Lab Assignment

1. Create a trigger to record employee salary updates.
2. Create a BEFORE DELETE trigger to prevent deletion of employees from the IT department.
3. Create an audit table for student records and implement INSERT, UPDATE, and DELETE triggers.
4. Create a trigger that automatically stores the current date during record insertion.
5. Create a trigger to validate employee age before insertion.
6. Compare BEFORE and AFTER triggers.

7. Explain row-level and statement-level triggers with examples.
 8. Write a trigger using both :OLD and :NEW values.
 9. Implement a trigger to prevent duplicate salary entries.
 10. Explain the practical applications of database triggers.
-

Precautions

- Test triggers on sample data before using them in production.
 - Avoid creating multiple triggers for the same event unless necessary.
 - Use meaningful trigger names.
 - Handle exceptions properly using RAISE_APPLICATION_ERROR.
 - Verify trigger execution using test DML operations.
 - Remember that triggers execute automatically and should not contain unnecessary complex logic.
-

Learning Outcome

After completing this experiment, students will be able to:

- Understand the concept and working of database triggers.
- Create BEFORE and AFTER triggers for INSERT, UPDATE, and DELETE events.
- Use :OLD and :NEW pseudo-records in row-level triggers.
- Implement audit logging and data validation using triggers.
- Apply triggers to automate database operations and enforce business rules in real-world applications.